

Ex: No: 10	Implementation of Code Optimization Phase of a Compiler
Date:	

Aim

To implement the code optimization phase of a compiler that eliminates redundant computations by identifying common subexpressions and replacing them with previously computed values.

Algorithm

Start the program.

Read the number of expressions.

Read each expression in three-address form.

Compare the expressions.

Identify statements containing the same operands and operator.

Keep the first occurrence of a common subexpression.

Replace subsequent occurrences with the previously computed result.

Display the optimized code.

Stop the program.

Program

```
#include <stdio.h>
#include <string.h>

struct code
{
    char lhs[10], op1[10], op2[10], op[5];
} c[20];

int main()
{
    int n, i, j;
    int flag[20] = {0};

    printf("Enter number of expressions: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("Enter expression %d (lhs op1 operator op2): ",
              i + 1);

        scanf("%9s %9s %4s %9s",
              c[i].lhs, c[i].op1, c[i].op, c[i].op2);
    }

    printf("\n--- Optimized Code ---\n");

    for (i = 0; i < n; i++)
    {
        if (flag[i] == 0)
        {
            printf("%s = %s %s %s\n",
                  c[i].lhs, c[i].op1, c[i].op, c[i].op2);

            for (j = i + 1; j < n; j++)
            {
                if (strcmp(c[i].op1, c[j].op1) == 0 &&
                    strcmp(c[i].op2, c[j].op2) == 0 &&
                    strcmp(c[i].op, c[j].op) == 0)
                {
                    printf("%s = %s\n",
                          c[j].lhs, c[i].lhs);
                }
            }
        }
    }
}
```

```
        flag[j] = 1;
    }
}
return 0;
}
```

Procedure to Execute

Step 1: Save the Program

Save the program as `optimization.c`.

Step 2: Open PowerShell

Navigate to the experiment folder: `cd E:\PCD1\exp10`.

Step 3: Compile

```
gcc optimization.c -o optimization.exe
```

Step 4: Run

```
.\optimization.exe
```

Sample Input

```
Enter number of expressions: 4
Enter expression 1 (lhs op1 operator op2): a b + c
Enter expression 2 (lhs op1 operator op2): d b + c
Enter expression 3 (lhs op1 operator op2): e a * f
Enter expression 4 (lhs op1 operator op2): g b + c
```

Output

```
--- Optimized Code ---
a = b + c
d = a
g = a
e = a * f
```

Explanation

The expression `b + c` occurs repeatedly. The program keeps its first computation as `a = b + c` and replaces the repeated computations with `d = a` and `g = a`. This demonstrates Common Subexpression Elimination (CSE).

Result

Thus, common subexpressions were identified and redundant computations were replaced with previously computed values, reducing repeated calculations.